

CLASSIC PACMAN GAME

A MINI PROJECT REPORT

Submitted by

ELAVARASAN P	(963220104016)
PATTU SUDAR U	(963220104038)
RAJA S	(963220104311)
SACHIN B	(963220104313)

In partial fulfilment for the award of the degree

Of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

PET ENGINEERING COLLEGE, VALLIOOR

ANNA UNIVERSITY : CHENNAI 600 025

MAY 2023

ABSTRACT

This paper is about implementing Classic PacMan games in javascript with the help of games library and search algorithms. Pac-Man is a 1980 maze video game created and released by Namco which was developed for arcades. The original Japanese title of Puck Man has been changed to Pac-Man for international release as a precautionary measure against the destruction of arcade machines by converting P into F. The player controls Pac-Man, who must eat all the dots inside the closed maze while avoiding the four colored ghosts.

The main aim is to build an intelligent pacman game in which the spirits/ ghosts which are present in the maze finds optimal paths through the maze to find a particular goal such as a finding shortest distance from current position of spirit to the position where pacman i.e player is heading in the maze .For that, we have implemented AI search algorithms like A*search,Uniform cost search. For implementing the project we have made use of the game library of javascript which is a cross-platform set of Javascript modules designed for writing video games. It includes computer graphics and sound libraries designed to be used with the Javascript programming language.

This paper describes the technical details and implementation of a Classic PacMan game . A small demo example and javascript modules which are created during creating a game are presented in the implementation section, to show the implementation of the project. Keywords: Pac Man; Maze; A*Search;Uniform Cost Search.

TABLE OF CONTENTS

CHAPTER. NO	TITLE	Pg. No
	ABSTRACT	i
1	INTRODUCTION	1
2	LITERATURE SURVEY	2
3	SYSTEM ANALYSIS	5
	3.1. Project initiation	5
	3.2. Planing	5
	3.3. Development	6
	3.4. Release	6
4	SYSTEM DESIGN	7
	4.1. Architecture diagram	7
	4.2. Sequence diagram	8
5	SYSTEM TESTING	9
6	RESULT & CONCLUSION	11
	6.1. Result	11
	6.2. Conclusion	12
7	APPENDIX	13
8	REFERENCE	32

1. INTRODUCTION

Game development began in early 1979, under the direction of Toru Iwatani and a team of nine men. Iwatani wanted to make a play that would not attract women and men, because most video games at that time had military or sports themes. Although the Pac-Man character's inspiration was the Pac-Man image, sliced pizza, Iwatani said he also incorporated a Japanese word meaning mouth, chuchi. The characters inside the game are made beautiful and colorful to attract younger players. Puckman's first Japanese title was taken from the hockey-puck form of the main character.

Pac-Man is an action maze chase video game; the player controls the eponymous character through an enclosed maze. The objective of the game is to eat all of the dots placed in the maze while avoiding four colored ghosts or spirits that pursue him. When Pac-Man eats all of the dots, the player advances to the next level. If Pac-Man makes contact with a ghost, he will lose a life; the game ends. But if pacman eats up all the dots in the maze the player is the winner.

Pac-man is known to be one of the most popular arcade games in the world. The inspiration for this project comes from our love of video games. Pacman Game in javascript is a simple console or small clip game designed for entertainment purposes. Pacman must be driven in such a way as to navigate the previously defined blue path in order to clear or be eaten by Pacman. When you clear a path, the more points you get; the game is easy to play.

The game will be designed in javascript programming language. The player must control the pacman of the arrow keys on his keyboard. If the Pac-man eats large dots it will turn the ghost blue which allows Pac-man to gain bonus points by eating them. To win the level, Pacman must eat all the food in the maze. Pacman will eat the food when he is done with it. If the Pac-man is captured by a ghost, then the game gets over. The inspiration for choosing the PAC MAN game comes from our passion for playing video games. In recent years video games have become quite popular. Which have improved the gaming industry. Pac man games will help young children improve their concentration and concentration levels in a fun way.

2. LITERATURE SURVEY

In the following literature review we went through multiple research papers. Going through the paper we came to know many concepts related to the pac man game. We examine several key approaches for describing the composition of games and comparison of different search algorithms for the . Also how the game can be implemented in 3d version unlike the B&W retro game on television. Finally we also mentioned other notable research papers related to the project.

In this paper [1], “Design of Pac-Man” implemented by Ke, Eric and Winston developed this project for reconstructing the Pac-Man game. In this, the project is split into software and hardware programming, and they have reduced hardware complexity with little effort on the software side. In this project, Game logic like collision detection, scoring and enemy artificial intelligence (AI) are included and are implemented using the MicroBlaze. It implements a graphics engine with sprites and paddle control over the MicroBlaze UART IP Core and Hardware configuration is described in Very Large Scale Integration HDL (VHDL).

In this paper [2], “The Pac-Man project” by Raymond Chen and John Martin L was to replicate the great arcade classic Pac-Man on television in black and white in a 3-d Version which can be used on any television inputs. This project used an Atmel AT90S8515 microcontroller and a simple video DAC to produce our Pac-Man game on any television with composite inputs.

In this paper [3], “Pacman Game Using Java”, they used a recursive Backtracking algorithm used in game theory. This project uses Java to discuss the popular Pacman game. With the maze, a Pac-man, and Pac-dots for the Pacman to eat, they created a simple Pacman implementation. They made a total of eight Pac-man images for this, one of which changes direction and has an alternating open/closed mouth. The Pac-man consumes the Pac-dots by moving around in legal positions at random. Several ghosts were added. This second increment was accomplished by

checking to see if the game has ended (ghost catches Pacman or Pacman consumes all of the dots). Although the graphical representation is the most visible aspect of the game, it is implemented utilizing multi-threading and synchronized methods and algorithms from Data Structures.

In this paper [4] “Artificial Intelligence based Pacman Game”, The goal was to create a Pac-man game using AI. Pac-Man is a difficult video game that can be helpful in AI (Artificial Intelligence) research. They have created several AI algorithms for the pacman game because it aids in the study of AI through the use of visualizations, which allows them to understand AI more effectively. The basic purpose is to create an intelligent Pacman agent that determines the best paths through the maze to achieve a specific goal, such as finding a specific food spot or avoiding ghosts. They've used AI search algorithms including Depth first search, Breadth first search, A*search, and Uniform cost search to accomplish this. Multi-agents such as Reflex agent, Minimax agent, and Alpha-beta agent have also been implemented. They can make Pacman react to its surroundings and escape from ghosts using these multiagent methods.

In this paper [5] this study proposes a model-based approach for computing real-time optimal choice strategies. Ms. Pac-Man is a great benchmark problem for a chase-evasion game with several, active adversaries that adjust their pursuit policies in response to Ms. Pac-condition Man's and decisions. The agent must chase various fixed and movable targets in an obstacle-filled environment in addition to avoiding the enemies. This work proposes a novel method for deriving a decision-tree representation of all feasible strategies from the maze geometry and the adversaries' or ghosts' dynamic equations. Extensive numerical simulations are used to validate the suggested models of ghost dynamics and decisions. The decision tree is updated and utilized to find optimal strategies throughout the game.

In this paper [6] “Learning to Play Pac-Man: An Evolutionary, Rule-based Approach”, this paper describes the first method of development, a man-made agent who plays the simplified version of Pac-Man. The agent is specified as a simple limited position machine and set of rules, with limits that control the probability of movement by the agent is given maze limits instantly time. Contrary to previous methods, the

agent stands for evolving Pac-Man gaming strategy, instead of a pre-arranged maze solution. The agent "reads" flexibly by adjusting agent parameters via population-based incremental learning (PBIL). Test results are available that provide insight into some of the complexities of the game, and highlight the limitations as well the difficulty of agent representation.

3. SYSTEM ANALYSIS

Software model used in the project is the agile model. Agile refers to something that is quick or adaptable. A software development approach based on iterative development is referred to as an "agile process model. Agile approaches divide projects into smaller iterations or sections and avoid long-term planning. The scope and requirements of the project are defined at the start of the development phase. The number of iterations, duration, and scope of each iteration are all clearly determined ahead of time. As we used an agile model the project was divided into smaller tasks which were like building stones for the project. Completing each task made little progress in the project. It was easy to do projects according to task as each concept was cleared in the particular tasks.

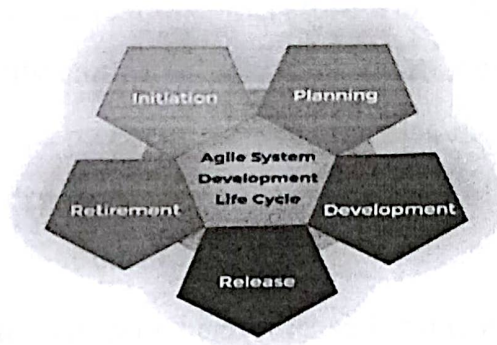


Fig 3.1: Phases of Agile model

3.1. Project Initiation: The beginning of the agile software development life cycle. This initial stage, also known as the inception or envisage phase, focuses on discussing the project vision. This is a high-level discussion of feasibility that does not get into specifics. This step entails identifying team members as well as determining the amount of time and work resources required to accomplish the project. Taking inventory of resources is critical for establishing the economic feasibility of a project before it can be approved.

3.2. Planning: The Agile lifecycle takes shape for the team during this exploratory phase. We talk about how growing the backlog at the story level will help us do this.

The type of user, what they want from the product, and why should all be included in the story. It's important to think about the business possibility in a larger framework. This will have an impact on the project's functional and financial viability. With an initial release strategy, you should estimate the risks and set milestones. Only when your backlog is complete and you've prioritized the items based on business value and dependency is planning complete.

3.3. Development: The actual work begins after the requirements have been developed based on feedback from the product owners and stakeholders. In incremental phases, sprints, or iterations, agile product development produces high-quality working products. Developers begin work on the first version of the product with the goal of completing the sprint with a working, usable product. This is not the final version and will go through several modifications, therefore it should only offer the bare minimum of features. This capability can be enhanced in future Agile lifecycle iterations. Teams can succeed in these sprints by ensuring team and stakeholder collaboration. Following code rules and style requirements to maintain quality. Following the priorities stated by stakeholders with complete control

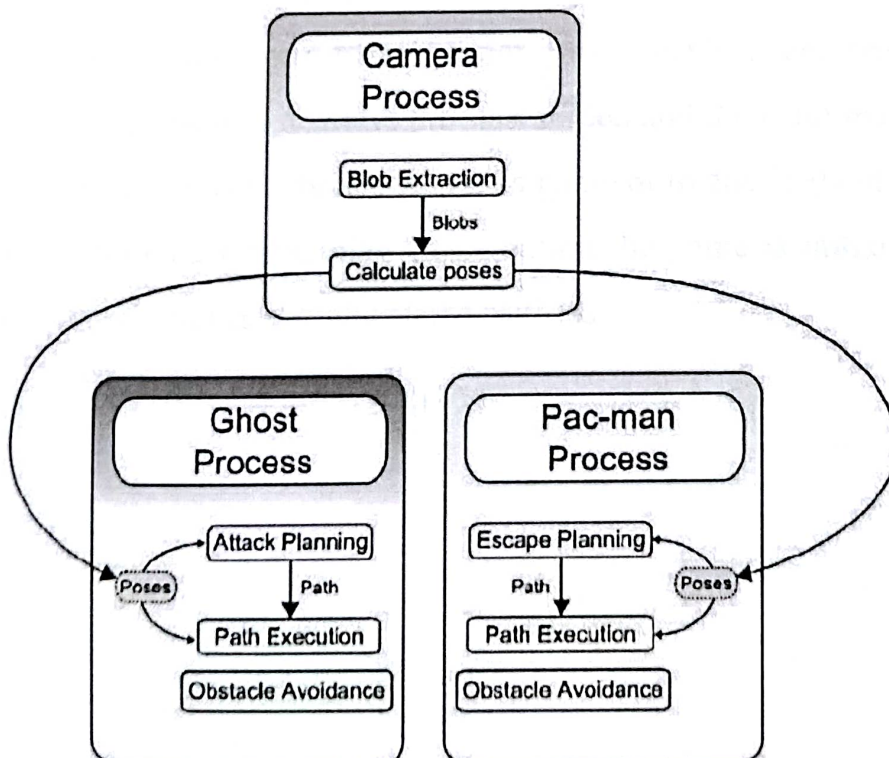
3.4. Release: Your product has now been released and is in use by end users. It's critical to keep an eye on these early phases for faults or defects that were missed during testing. Between the production and support teams, there should be a handover with appropriate training. Depending on the type of product you're producing, these final processes and handovers may differ. When the product is ready to be retired, the production phase usually finishes.

4. SYSTEM DESIGN

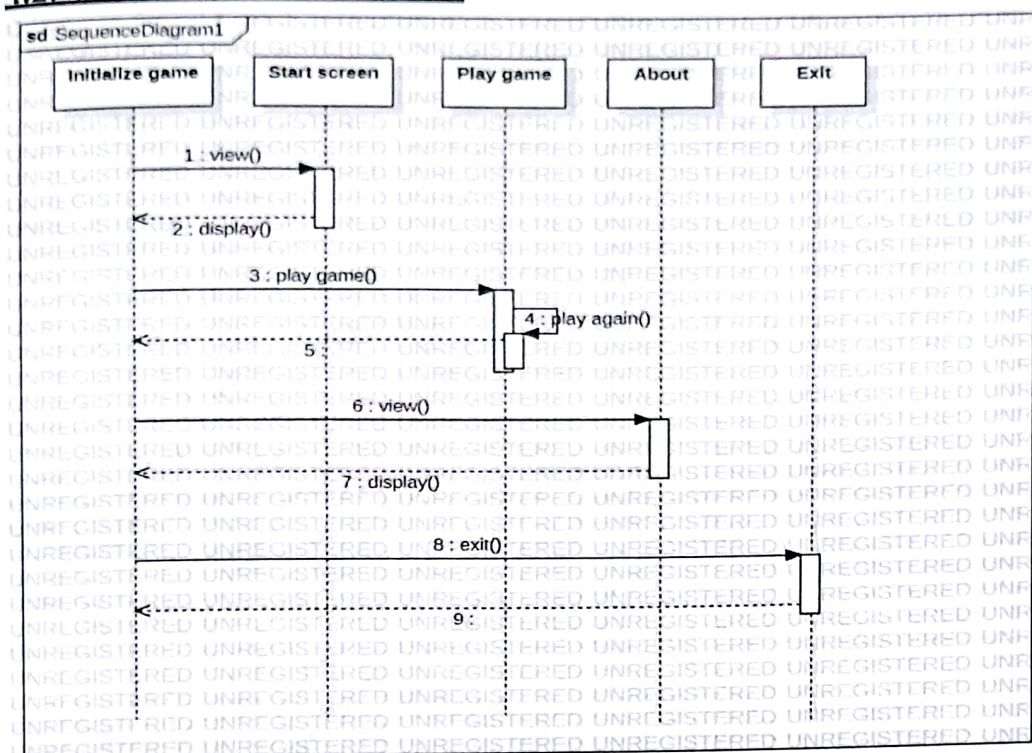
Objective of the project is to create a maze inside which game is played The goal of the game is to eat all of the dots placed in the maze while avoiding four colored ghosts or spirits that pursue him. Create a pacman which is controlled by player using arrow keys When Pac-Man eats all of the dots, the player wins the game .If Pac-Man makes contact with a ghost, he will lose a life the game ends .To improve the overall user experience

In the era of AAA and high speed games with amazing high graphics we wanted to create a legendary arcade game pac man which is famous from many decades but the new generation have started to forget the game. In addition to classic retro games we have tried to add better graphics and made it a bit easy to play for the new generation. There are many different versions of pac man game but we created a Classic version of pac man Our game version has a pac man that is the player itself and 4 ghosts which are named as Blinky, Pinky, Inky and Clyde collectively known as Ghost Gang. Game is kept quite easy to play so that even younger children can enjoy playing it.

4.1.ARCHITECTURE DIAGRAM

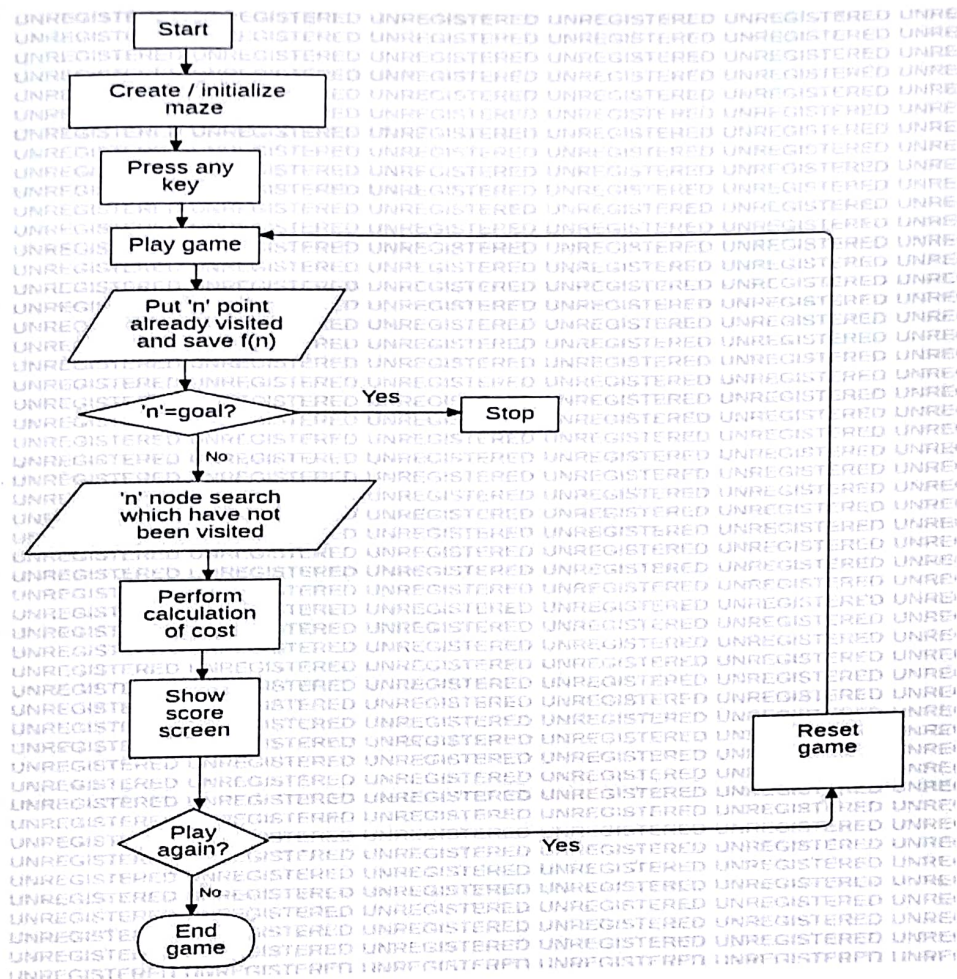


4.2. SEQUENCE DIAGRAM



Sequence diagrams depict how items interact with one another and in what sequence they do so. A sequence diagram depicts item interactions in chronological order. It illustrates the scenario's objects and classes, as well as the sequence of messages sent between them in order to carry out the scenario's functionality. The lifeline is a line that depicts how long an object has been present in a process. After the game is initialized it shows the start screen and the main menu is displayed. In which a player can choose whether to play a game or to check about the game or exit the game. After clicking on the play game button the game is initialized and the pac man and ghost are initialized at the start positions.

5. SYSTEM TESTING



The two search algorithms which we studied are Uniform Cost search algorithm and A* Algorithms.

A* Search: An educated search algorithm with prior knowledge of the problem is known as an A* search. A* is a best-first-search algorithm using the formula $f(n) = g(n) + h(n)$, where $f(n) = g(n) + h(n)$.

$g(n)$ = total cost of edges from start to n

$h(n)$ = lowest-cost path from n to goal

estimate $f(n)$ = current distance +

estimated remaining distance

If $h(n)$ underestimates the cost of any solution that can be obtained from n, it is said to be acceptable. If $C^*(n)$ is the cheapest solution path from n to a destination node, and h is permissible, then $h(n) \leq C^*(n)$

If $h(n)$ is admissible, we can show that the search will find an optimal solution. [8].

OPEN = nodes on frontier.

CLOSED = expanded nodes. OPEN =

$\{ \langle s, \text{nil} \rangle \}$ while OPEN is not empty

remove from OPEN the node $\langle n, p \rangle$ with

minimum $f(n)$ place $\langle n, p \rangle$ on CLOSED

if n is a goal node, return success (path p)

for each edge connection n & m with cost

c

if $\langle m, q \rangle$ is on CLOSED and $\{p|e\}$ is

cheaper than q then remove n from

CLOSED, put $\langle m, \{p|e\} \rangle$ on OPEN

else if $\langle m, q \rangle$ is on OPEN and $\{p|e\}$ is

cheaper than q then replace q with

$\{p|e\}$ else if m is not on OPEN then

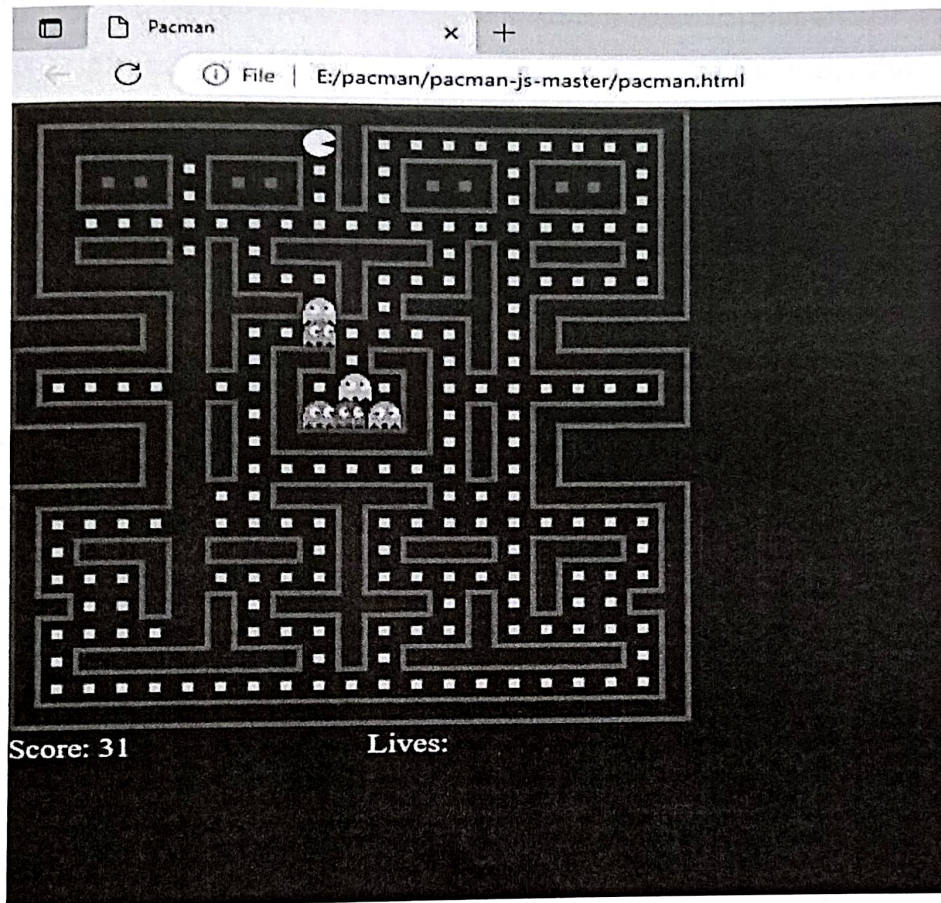
put $\langle m, \{p|e\} \rangle$ on OPEN return failure.

Uniform Cost Search: In uniform cost search we enqueue nodes by path cost. Let $g(n)$ = cost of the path from the start node to the current node n . The algorithm sorts nodes by increasing the value of g , and expands the lowest cost node of the fringe. Properties of Uniform Cost Search

- Complete
- Optimal/Admissible
- Exponential time and space complexity, $O(b^d)$

6. RESULT & CONCLUSION

6.1 Result



and have a strategy in solving the problem.

In conclusion, Games can be described as a very big thing. The game is made to give a bit of fun always play creatively and do not abuse the game. The children who play the games can improve their concentration and have a strategy in solving the problem. It is quite easy and interesting so it won't be boring for younger children. We have made use of library to give the screen so it's interesting which was the goal of our original project too.

6.2 Conclusion

The primary goal of the project was to create a Retro Classic pacman game for the purpose of entertainment and which can be played by all age groups. Pac man which finds its origin back in 1980 created in Japan is one of the most famous arcade games. The project which we undertook has helped us to gain a better perspective on various aspects related to our course of study as well as particular knowledge of particular cmd-based applications.

According to this project, we understand some of the key algorithms, heuristic search, and specific game theory. Based on this information, we use python to design two different agents. One can be a pacman and eat the food of rivals, and one can always be a defensive ghost, preventing the pacman from eating our food. Basically we use two different algorithms for this project, A* search and UCF. Each algorithm has its advantages and disadvantages. Finally we selected A* because of its less time complexity and its more efficient in finding the goal node.

The player who succeeds in eating more pills by avoiding ghosts scores many points. Game can be won when pacman eats up all the food pills without getting killed by 4 ghosts. The children who play the pacman game can improve their concentration and have a strategy in solving the problem.

In conclusion, Games can be described as a very big thing. The game is able to make his children always play creatively and do not abuse the game. The children who play the pacman game can improve their concentration and have a strategy in solving the problem. It is kept easy and interesting so it won't be boring for younger children. We have made use of vibrant colors so the screen seems interesting which was the goal of the original project too.

7. APPENDIX

7.1 PACMAN.HTML

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Pacman</title>
```

```
</head>
```

```
<body style="margin: 0; background-color: black;">
```

```
<canvas id="canvas" width="500" height="500"></canvas>
```

```
<div style="display:none;">
```

```

```

```

```

```
</div>
```

```
<script src="ghost.js"></script>
```

```
<script src="pacman.js"></script>
```

```
<script src="game.js"></script>
```

```
</body>
```

```
</html>
```

7.2 GAME.JS

```
const canvas = document.getElementById("canvas");
```

```
const canvasContext = canvas.getContext("2d");
```

```
const pacmanFrames = document.getElementById("animation");
```

```
const ghostFrames = document.getElementById("ghosts");
```

```
let createRect = (x, y, width, height, color) => {
```

```
    canvasContext.fillStyle = color;
```

```
    canvasContext.fillRect(x, y, width, height);
```

```
};
```

```
const DIRECTION_RIGHT = 4;
```

```
const DIRECTION_UP = 3;
```

```
const DIRECTION_LEFT = 2;
```

```
const DIRECTION_BOTTOM = 1;
```

```
let lives = 3;
```

```
let ghostCount = 4;
```

```
let ghostImageLocations = [
```

```
    { x: 0, y: 0 },
```

```
    { x: 176, y: 0 },
```



```

    { x: 0, y: 121 },
    { x: 176, y: 121 },
];

// Game variables

let fps = 30;

let pacman;

let oneBlockSize = 20;

let score = 0;

let ghosts = [];

let wallSpaceWidth = oneBlockSize / 1.6;

let wallOffset = (oneBlockSize - wallSpaceWidth) / 2;

let wallInnerColor = "black";

// we now create the map of the walls,

// if 1 wall, if 0 not wall

// 21 columns // 23 rows

let map = [

    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
    [1, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1],
    [1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 1, 2, 1, 1, 1, 2],
    [1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 1, 2, 1, 1, 1, 2],
    [1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1],
    [1, 2, 1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 1, 2, 1, 2, 1, 1, 1, 2],
    [1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 2, 1],

```

```

[1, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 1, 1, 1],
[0, 0, 0, 0, 1, 2, 1, 2, 2, 2, 2, 2, 2, 1, 2, 1, 0, 0, 0, 0],
[1, 1, 1, 1, 1, 2, 1, 2, 1, 1, 2, 1, 1, 2, 1, 2, 1, 1, 1, 1],
[1, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1],
[1, 1, 1, 1, 1, 2, 1, 2, 1, 2, 2, 2, 1, 2, 1, 2, 1, 1, 1, 1],
[0, 0, 0, 0, 1, 2, 1, 2, 1, 1, 1, 1, 1, 2, 1, 2, 1, 0, 0, 0, 0],
[0, 0, 0, 0, 1, 2, 1, 2, 2, 2, 2, 2, 2, 2, 1, 2, 1, 0, 0, 0, 0],
[1, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1, 1, 2, 2, 2, 1, 1, 1, 1],
[1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 1],
[1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1],
[1, 2, 2, 2, 1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1],
[1, 1, 2, 2, 1, 2, 1, 2, 1, 1, 1, 1, 1, 2, 1, 2, 1, 2, 2, 1, 1],
[1, 2, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 2, 2, 1],
[1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1],
[1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1],
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
];

```

```

let randomTargetsForGhosts = [
  { x: 1 * oneBlockSize, y: 1 * oneBlockSize },
  { x: 1 * oneBlockSize, y: (map.length - 2) * oneBlockSize },
  { x: (map[0].length - 2) * oneBlockSize, y: oneBlockSize },
  {
    x: (map[0].length - 2) * oneBlockSize,
    y: (map.length - 2) * oneBlockSize,
  }
];

```

```

    },
];

// for (let i = 0; i < map.length; i++) {
//   for (let j = 0; j < map[0].length; j++) {
//     map[i][j] = 2;
//   }
// }

```

```

let createNewPacman = () => {
  pacman = new Pacman(
    oneBlockSize,
    oneBlockSize,
    oneBlockSize,
    oneBlockSize,
    oneBlockSize / 5
  );
};

```

```

let gameLoop = () => {
  update();
  draw();
};

```

```

let gameInterval = setInterval(gameLoop, 1000 / fps);

```



```

let restartPacmanAndGhosts = () => {
  createNewPacman();
  createGhosts();
};

```

```

let onGhostCollision = () => {
  lives--;
  restartPacmanAndGhosts();
  if (lives === 0) {
  }
};

```

```

let update = () => {
  pacman.moveProcess();
  pacman.eat();
  updateGhosts();
  if (pacman.checkGhostCollision(ghosts)) {
    onGhostCollision();
  }
};

```

```

let drawFoods = () => {
  for (let i = 0; i < map.length; i++) {
    for (let j = 0; j < map[0].length; j++) {
      if (map[i][j] === 2) {

```

```

    createRect(
      j * oneBlockSize + oneBlockSize / 3,
      i * oneBlockSize + oneBlockSize / 3,
      oneBlockSize / 3,
      oneBlockSize / 3,
      "#FEB897"
    );
  }
}
};

```

```

let drawRemainingLives = () => {
  canvasContext.font = "20px Emulogic";
  canvasContext.fillStyle = "white";
  canvasContext.fillText("Lives: ", 220, oneBlockSize * (map.length + 1));

  for (let i = 0; i < lives; i++) {
    canvasContext.drawImage(
      pacmanFrames,
      2 * oneBlockSize,
      0,
      oneBlockSize,
      oneBlockSize,
      350 + i * oneBlockSize,

```

```

    oneBlockSize * map.length + 2,

    oneBlockSize,

    oneBlockSize

  );

}

};

```

```

let drawScore = () => {

  canvasContext.font = "20px Emulogic";

  canvasContext.fillStyle = "white";

  canvasContext.fillText(

    "Score: " + score,

    0,

    oneBlockSize * (map.length + 1)

  );

};

```

```

let draw = () => {

  canvasContext.clearRect(0, 0, canvas.width, canvas.height);

  createRect(0, 0, canvas.width, canvas.height, "black");

  drawWalls();

  drawFoods();

  drawGhosts();

  pacman.draw();

  drawScore();

```

```

drawRemainingLives();

};

let drawWalls = () => {

  for (let i = 0; i < map.length; i++) {

    for (let j = 0; j < map[0].length; j++) {

      if (map[i][j] == 1) {

        createRect(

          j * oneBlockSize,

          i * oneBlockSize,

          oneBlockSize,

          oneBlockSize,

          "#342DCA"

        );

        if (j > 0 && map[i][j - 1] == 1) {

          createRect(

            j * oneBlockSize,

            i * oneBlockSize + wallOffset,

            wallSpaceWidth + wallOffset,

            wallSpaceWidth,

            wallInnerColor

          );

        }

        if (j < map[0].length - 1 && map[i][j + 1] == 1) {

```



```

createRect(
    j * oneBlockSize + wallOffset,
    i * oneBlockSize + wallOffset,
    wallSpaceWidth + wallOffset,
    wallSpaceWidth,
    wallInnerColor
);
}

```

```

if (i < map.length - 1 && map[i + 1][j] == 1) {

```

```

    createRect(
        j * oneBlockSize + wallOffset,
        i * oneBlockSize + wallOffset,
        wallSpaceWidth,
        wallSpaceWidth + wallOffset,
        wallInnerColor
    );
}

```

```

if (i > 0 && map[i - 1][j] == 1) {

```

```

    createRect(
        j * oneBlockSize + wallOffset,
        i * oneBlockSize,
        wallSpaceWidth,
        wallSpaceWidth + wallOffset,

```

```

        wallInnerColor
    );
}
}
}
}
};

```

```

let createGhosts = () => {
    ghosts = [];
    for (let i = 0; i < ghostCount * 2; i++) {
        let newGhost = new Ghost(
            9 * oneBlockSize + (i % 2 == 0 ? 0 : 1) * oneBlockSize,
            10 * oneBlockSize + (i % 2 == 0 ? 0 : 1) * oneBlockSize,
            oneBlockSize,
            oneBlockSize,
            pacman.speed / 2,
            ghostImageLocations[i % 4].x,
            ghostImageLocations[i % 4].y,
            124,
            116,
            6 + i
        );
        ghosts.push(newGhost);
    }
}

```

```

};

createNewPacman();

createGhosts();

gameLoop();

window.addEventListener("keydown", (event) => {

  let k = event.keyCode;

  setTimeout(() => {

    if (k == 37 || k == 65) {

      // left arrow or a

      pacman.nextDirection = DIRECTION_LEFT;

    } else if (k == 38 || k == 87) {

      // up arrow or w

      pacman.nextDirection = DIRECTION_UP;

    } else if (k == 39 || k == 68) {

      // right arrow or d

      pacman.nextDirection = DIRECTION_RIGHT;

    } else if (k == 40 || k == 83) {

      // bottom arrow or s

      pacman.nextDirection = DIRECTION_BOTTOM;

    }

  }, 1);

});

```

7.3 GHOST.JS

```
class Ghost {
  constructor(
    x,
    y,
    width,
    height,
    speed,
    imageX,
    imageY,
    imageWidth,
    imageHeight,
    range
  ) {
    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
    this.speed = speed;
    this.direction = DIRECTION_RIGHT;
    this.imageX = imageX;
    this.imageY = imageY;
    this.imageHeight = imageHeight;
    this.imageWidth = imageWidth;
    this.range = range;
    this.randomTargetIndex = parseInt(Math.random() * 4);
    this.target = randomTargetsForGhosts[this.randomTargetIndex];
    setInterval(() => {
      this.changeRandomDirection();
    }, 10000);
  }

  isInRange() {
    let xDistance = Math.abs(pacman.getMapX() - this.getMapX());
    let yDistance = Math.abs(pacman.getMapY() - this.getMapY());
    if (
      Math.sqrt(xDistance * xDistance + yDistance * yDistance) <=
        this.range
    ) {
      return true;
    }
  }
}
```



```

    return false;
}

changeRandomDirection() {
    let addition = 1;
    this.randomTargetIndex += addition;
    this.randomTargetIndex = this.randomTargetIndex % 4;
}

moveProcess() {
    if (this.isInRange()) {
        this.target = pacman;
    } else {
        this.target = randomTargetsForGhosts[this.randomTargetIndex];
    }
    this.changeDirectionIfPossible();
    this.moveForwards();
    if (this.checkCollisions()) {
        this.moveBackwards();
        return;
    }
}

moveBackwards() {
    switch (this.direction) {
        case 4: // Right
            this.x -= this.speed;
            break;
        case 3: // Up
            this.y += this.speed;
            break;
        case 2: // Left
            this.x += this.speed;
            break;
        case 1: // Bottom
            this.y -= this.speed;
            break;
    }
}

moveForwards() {
    switch (this.direction) {
        case 4: // Right

```

```

    this.x += this.speed;
    break;
case 3: // Up
    this.y -= this.speed;
    break;
case 2: // Left
    this.x -= this.speed;
    break;
case 1: // Bottom
    this.y += this.speed;
    break;
}
}

checkCollisions() {
    let isCollided = false;
    if (
        map[parseInt(this.y / oneBlockSize)][
            parseInt(this.x / oneBlockSize)
        ] = 1 ||
        map[parseInt(this.y / oneBlockSize + 0.9999)][
            parseInt(this.x / oneBlockSize)
        ] = 1 ||
        map[parseInt(this.y / oneBlockSize)][
            parseInt(this.x / oneBlockSize + 0.9999)
        ] = 1 ||
        map[parseInt(this.y / oneBlockSize + 0.9999)][
            parseInt(this.x / oneBlockSize + 0.9999)
        ] = 1
    ) {
        isCollided = true;
    }
    return isCollided;
}

changeDirectionIfPossible() {
    let tempDirection = this.direction;
    this.direction = this.calculateNewDirection(
        map,
        parseInt(this.target.x / oneBlockSize),
        parseInt(this.target.y / oneBlockSize)
    );
    if (typeof this.direction === "undefined") {

```

```

    this.direction = tempDirection;
    return;
}
if (
    this.getMapY() != this.getMapYRightSide() &&
    (this.direction == DIRECTION_LEFT ||
    this.direction == DIRECTION_RIGHT)
) {
    this.direction = DIRECTION_UP;
}
if (
    this.getMapX() != this.getMapXRightSide() &&
    this.direction == DIRECTION_UP
) {
    this.direction = DIRECTION_LEFT;
}
this.moveForwards();
if (this.checkCollisions()) {
    this.moveBackwards();
    this.direction = tempDirection;
} else {
    this.moveBackwards();
}
console.log(this.direction);
}

```

```

calculateNewDirection(map, destX, destY) {

```

```

    let mp = [];
    for (let i = 0; i < map.length; i++) {
        mp[i] = map[i].slice();
    }

```

```

    let queue = [
        {
            x: this.getMapX(),
            y: this.getMapY(),
            rightX: this.getMapXRightSide(),
            rightY: this.getMapYRightSide(),
            moves: [],
        },
    ];

```

```

    while (queue.length > 0) {
        let popped = queue.shift();
    }

```

```

    if (poped.x === destX && poped.y === destY) {
        return poped.moves[0];
    } else {
        mp[poped.y][poped.x] = 1;
        let neighborList = this.addNeighbors(poped, mp);
        for (let i = 0; i < neighborList.length; i++) {
            queue.push(neighborList[i]);
        }
    }
}

return 1; // direction
}

addNeighbors(poped, mp) {
    let queue = [];
    let numOfRows = mp.length;
    let numOfColumns = mp[0].length;

    if (
        poped.x - 1 >= 0 &&
        poped.x - 1 < numOfRows &&
        mp[poped.y][poped.x - 1] !== 1
    ) {
        let tempMoves = poped.moves.slice();
        tempMoves.push(DIRECTION_LEFT);
        queue.push({ x: poped.x - 1, y: poped.y, moves: tempMoves });
    }

    if (
        poped.x + 1 >= 0 &&
        poped.x + 1 < numOfRows &&
        mp[poped.y][poped.x + 1] !== 1
    ) {
        let tempMoves = poped.moves.slice();
        tempMoves.push(DIRECTION_RIGHT);
        queue.push({ x: poped.x + 1, y: poped.y, moves: tempMoves });
    }

    if (
        poped.y - 1 >= 0 &&
        poped.y - 1 < numOfColumns &&
        mp[poped.y - 1][poped.x] !== 1
    ) {
        let tempMoves = poped.moves.slice();

```

```

    tempMoves.push(DIRECTION_UP);
    queue.push({ x: popped.x, y: popped.y - 1, moves: tempMoves });
  }
  if (
    popped.y + 1 >= 0 &&
    popped.y + 1 < numOfColumns &&
    mp[popped.y + 1][popped.x] != 1
  ) {
    let tempMoves = popped.moves.slice();
    tempMoves.push(DIRECTION_BOTTOM);
    queue.push({ x: popped.x, y: popped.y + 1, moves: tempMoves });
  }
  return queue;
}

getMapX() {
  let mapX = parseInt(this.x / oneBlockSize);
  return mapX;
}

getMapY() {
  let mapY = parseInt(this.y / oneBlockSize);
  return mapY;
}

getMapXRightSide() {
  let mapX = parseInt((this.x * 0.99 + oneBlockSize) / oneBlockSize);
  return mapX;
}

getMapYRightSide() {
  let mapY = parseInt((this.y * 0.99 + oneBlockSize) / oneBlockSize);
  return mapY;
}

changeAnimation() {
  this.currentFrame =
    this.currentFrame == this.frameCount ? 1 : this.currentFrame + 1;
}

draw() {
  canvasContext.save();
  canvasContext.drawImage(

```



```

ghostFrames,
this.imageX,
this.imageY,
this.imageWidth,
this.imageHeight,
this.x,
this.y,
this.width,
this.height
);
// canvasContext.restore();
// canvasContext.beginPath();
//canvasContext.strokeStyle = "red";
// canvasContext.arc(
//  this.x + oneBlockSize / 2,
//  this.y + oneBlockSize / 2,
//this.range * oneBlockSize,
//0,
// 2 * Math.PI
// );
canvasContext.stroke();
}
}

let updateGhosts = () => {
  for (let i = 0; i < ghosts.length; i++) {
    ghosts[i].moveProcess();
  }
};

let drawGhosts = () => {
  for (let i = 0; i < ghosts.length; i++) {
    ghosts[i].draw();
  }
};

```

8. REFERENCE

- [1] Ke, Eric and Winston, "Design of Pac-Man," IEEE, 2014.
- [2] Raymond Chen and John Martin L, "The Pac- Man Project," IRJET Volume:04 Issue:2015.
- [3] Uthkarsh Ch., "Pac-man Game using Java," Kakatiya Institute of Technology & Science, Warangal(A.P.) 2018.
- [4] Himakar Sai Chowdary Maddipati, Aravind Kundurthi, Pothula Mahima Raaj, Kapudasi Srilatha, Ravi Kishen Surapaneni, "Artificial Intelligence based Pacman Game," International Journal of Innovative Technology and Exploring Engineering (IJITEE) ISSN: 2278-3075 (Online), Volume-9 Issue-9, July 2020.
- [5] Greg Foderaro, Ashleigh Swingler, Silvia Ferrari, "A Model-based Approach to Optimizing Ms. Pac-Man Game Strategies in Real Time," School of Information Technology and Electrical Engineering University of Queensland 4072 Australia 2016.
- [6] Marcus Gallagher, Amanda Ryan, "Learning to Play Pac-Man: An Evolutionary, Rule-based Approach," IEEE Transactions on Computational Intelligence and AI in Games DRAFT - JANUARY 27, 2016.
- [7] John DeNero, Dan Klein, "Pacman Project" UC Berkeley CS188 Intro to AI, 2015.
- [8] "Problem Solving using Search- Single agent search," Version 2 CSE IIT Kharagpur, 2018.